The Journal of Open Source Software

SED_Tools: A Unified Pipeline for Acquiring, Standardizing, and Interpolating Stellar Atmosphere Spectral Energy Distributions

Niall J. Miller[1] **and Meridith Joyce**[2]

1 REPLACE WITH INSTITUTION AND DEPARTMENT **2** REPLACE WITH INSTITUTION AND DEPARTMENT

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

SED_Tools is an open-source Python package that downloads, validates, and standardizes stellar atmosphere spectral energy distribution (SED) grids from heterogeneous public archives, and converts them into the binary flux-cube and lookup-table format required for rapid runtime interpolation by stellar evolution codes. It ships a command-line interface (sed-tools) and a matching Python API (SED, Catalog, Spectrum, Filters, CatalogInfo) that together cover the full pipeline: spectra acquisition, filter transmission-curve acquisition and combination, flux-cube construction and rebuilding, grid densification via interpolation, target-directory preparation for stellar evolution codes, coverage diagnostics, and import of externally sourced grids. Supported source archives include the Spanish Virtual Observatory (SVO), the MAST/BOSZ synthetic library (Mészáros et al., 2024), and the Munari Stellar Grid (MSG) collection, spanning model families such as Kurucz/ATLAS9 (Castelli & Kurucz, 2003), NextGen/NextGen2, BT-Settl, TLUSTY (OSTAR2002/BSTAR2006), TMAP, GRAMS, SPHINX, Husfeld, and blackbody references. Two PyTorch-based components extend the tool beyond format conversion: an SED completer that extends incomplete wavelength coverage, and an SED generator that synthesizes an SED directly from (Teff, log g, [M/H]) without requiring an existing grid node. All standardization funnels through two single-authority modules — a header parser and a unit-cleaning routine — that guarantee a consistent output representation regardless of the conventions used by the source archive.

Statement of need

Stellar evolution and synthetic-photometry codes need pre-computed grids of stellar atmosphere spectra spanning effective temperature, surface gravity, and metallicity. In practice, these grids are scattered across independent archives that differ in file formats, header-keyword conventions, flux and wavelength units, wavelength sampling, and axis conventions; some archives additionally encode extra physical axes — sedimentation efficiency, helium fraction, alpha enhancement — as separate files rather than as grid dimensions, which silently produces duplicate or colliding nodes if not accounted for. A researcher who wants to combine, for example, a hot-star TLUSTY grid with a cool-star BT-Settl grid for a single evolutionary track must hand-reconcile all of these inconsistencies before any interpolation is possible, and errors introduced during that reconciliation — unit mismatches, axis-order bugs, double-counted nodes — are difficult to detect after the fact and can silently bias downstream synthetic photometry rather than produce an obvious failure. This is not a hypothetical risk: during SED_Tools's own development, a read/write axis-order mismatch in the flux-cube binary format produced exactly this kind of silent corruption, one that would have propagated undetected into every downstream synthetic photometric calculation had it not been caught by dedicated validation. `SED_Tools` exists to remove this class of manual, error-prone reconciliation from the workflow entirely. It automates the download of raw grids and filter curves, validates each file against its expected physical ranges, resolves duplicate or colliding grid nodes introduced by hidden axes, and writes a single standardized product — as flat per-model spectral files, an HDF5 archive, and a pre-computed binary flux cube with an accompanying lookup table — that a stellar evolution code can consume directly, with no additional user-side reformatting. This directly serves the MESA `colors` module, whose documentation designates `SED_Tools` as the tool that prepares its required data products, but the standardized output is equally usable by any downstream synthetic photometry, spectral energy distribution fitting, or population synthesis workflow that needs a self-consistent, science-ready stellar atmosphere grid.

State of the field

Several existing tools address related but distinct problems. `synphot`/`pysynphot` (STScI) perform synthetic photometry given an already-prepared SED and bandpass, but do not download, standardize, or reconcile heterogeneous source archives into interpolation-ready grids. `specutils`, part of the Astropy ecosystem, provides general-purpose spectral-object handling and analysis but is not oriented toward multi-source stellar atmosphere grid curation or production of runtime-optimized binary grids for external codes; it operates one spectrum at a time rather than as a grid-construction pipeline. `gollum` (Shankar et al., 2024) provides programmatic access to specific precomputed synthetic grids (e.g. PHOENIX, Sonora) with an interactive comparison interface, but targets spectral inspection and model-data comparison rather than multi-archive standardization or production of evolution-code-ready binary grids. MSG ships its own compiled grid-access library together with a defined set of packaged grids, but does not provide a general mechanism for ingesting and standardizing grids sourced from other archives such as SVO or MAST/BOSZ — a user who wants an MSG-format grid built from BOSZ or SVO data has no supported path to do so without custom scripting. Archive-side services such as the SVO Filter Profile Service (Rodrigo & Solano, 2020) and MAST provide raw data access but explicitly leave unit standardization, header reconciliation, collision handling, and target-code packaging to the user. To the authors' knowledge, no existing public tool spans SVO, MAST/BOSZ, and MSG-hosted grids simultaneously behind a single collision- aware standardization layer. `SED_Tools`'s distinguishing contribution is treating grid acquisition, unit and header standardization, hidden-axis collision resolution, and stellar- evolution-code-ready packaging as a single reproducible, versioned pipeline spanning multiple source archives at once, rather than as a one-off per-grid conversion script maintained privately by individual research groups. The machine-learning-based SED completion and generation components are also, to the authors' knowledge, not offered by any comparable publicly available tool in this space.

Software design

`SED_Tools` separates concerns along two axes: a data-flow axis and an interface axis. On the data-flow axis, two modules act as sole authorities over specific transformations, so that behavior does not depend on call order or on which grid is processed first. A header-parsing module resolves the many historical header-key spellings used across archives against a single alias table, using a nan-guard so that a later, unparseable value can never silently overwrite an already-resolved one. A spectra-cleaning module performs the one-time conversion of every spectrum to a common wavelength/flux unit system (angstroms; erg s ^{1} cm ^{2} Å ^{1}), after which no downstream stage is permitted to renormalize — this constraint is enforced by design rather than by convention, so that a unit bug can only be introduced, and only needs to be fixed, in one place. Hidden physical axes that some archives encode as separate files rather than as grid dimensions are resolved by a dedicated collision-handling module that builds both a mean flux cube and a library of per-variant cubes, so that information distinguishing the colliding variants is preserved rather than discarded during standardization; the on-collision strategy (average everything into one cube, keep every variant, or filter to a specific variant) is a per-model, reproducible configuration rather than an implicit default. Large grids — the BT-Settl grid alone occupies roughly 174 GiB once expanded to a common wavelength grid — are handled via memory-mapped array allocation for both source and destination flux arrays, rather than in-RAM allocation, specifically to avoid exhausting system memory during cube construction. On the interface axis, every capability is available identically through the `sed-tools` command-line interface and the Python `SED`/`Catalog`/`Spectrum` API, so the package is usable both as an interactive or scriptable library and as a stage in shell-based data pipelines. A dedicated coverage-diagnostics command lets a user inspect the completeness of a standardized grid in parameter space before relying on it for interpolation, and the ML-based completer and generator modules provide a fallback in regions where empirical grid coverage is insufficient. Error handling throughout the pipeline is intentionally non-defensive: malformed input is designed to raise immediately rather than be silently caught and skipped, so that data-quality problems surface at the point of ingestion rather than as a difficult-to-trace downstream artifact in a stellar model.

Correctness is checked at two levels. An automated `pytest` suite exercises the standardization utilities, configuration, CLI, and flux-cube loading and evaluation code paths directly. Separately, a dedicated audit pipeline performs physically motivated consistency checks on the standardized output itself: comparing each spectrum's flux-weighted peak wavelength against the Wien's-law prediction from its catalogued effective temperature, comparing bolometric flux against the blackbody integral over each file's actual wavelength coverage, and matching raw files against their corresponding flux-cube grid node under a tight distance tolerance to avoid false positives from molecular-band features. As of the most recent audit run, all installed models pass (12/12). The audit process has also been useful for separating genuine data problems from methodology artifacts: an initially suspected raw-versus-cube mismatch in the NextGen2 grid was traced to the comparison methodology itself (disagreement at molecular-band features introduced by interpolating onto a union wavelength grid) rather than to the data, and apparent Wien-peak violations in the Husfeld and tmap2 grids were confirmed to be a real, physically expected redward shift caused by UV/EUV atmospheric opacity in hot compact objects, not a unit error. Two smaller items remain open and are tracked rather than silently patched: a known $1/\pi$ flux-normalization discrepancy in the blackbody-reference generator, and a structural incompatibility between one MSG-hosted grid (`sg-BSTAR2006-low`) and the current HDF5 reader.

Research impact statement

The most concrete evidence of `SED_Tools`'s research impact is its role as the designated data-preparation pipeline for the MESA `colors` module, a new module merged into the official Modules for Experiments in Stellar Astrophysics (MESA) code base, first shipped in release candidate r25.10.1-rc1 and included in stable releases from r25.12.1 onward (Team, 2025). MESA is a widely used, actively maintained open-source stellar evolution code with a large user base spanning asteroseismology, stellar population synthesis, and binary evolution research (Jermyn et al., 2023; Paxton et al., 2011, 2013, 2015, 2018, 2019). The `colors` module's source documentation contains a dedicated data-preparation section naming `SED_Tools` as the tool that automates production of the module's inputs, and describes the pre-computed flux cube consumed by its fastest interpolation path as produced by `SED_Tools` (The MESA Team, 2026); the Kurucz2003 atmosphere grids distributed with the module were themselves prepared with `SED_Tools`. Every MESA user who enables synthetic photometry via the `colors` module is therefore, in practice, a downstream consumer of `SED_Tools`'s output, independent of whether they interact with the `SED_Tools` repository directly. This is a stronger and more durable form of research impact than citation count alone, because MESA's release and documentation infrastructure make the dependency independently verifiable rather than self-reported. As a secondary, softer indicator, the repository has accumulated organic community interest without any promotional effort.

AI usage disclosure

OpenAI Codex and Claude Code assisted with delinting/formatting during a codebase refactor, with constructing portions of the automated test suite, and with debugging flux-cube-construction performance issues; all such changes were reviewed and validated by the author before merging. All architectural decisions described in Software Design were made by the author, not the AI tools. This manuscript was drafted with Claude's assistance from author interviews and reviewed and edited by the author before submission.

Acknowledgements

REPLACE: list any funding, grants, or institutional support here, if applicable. Leave this section out entirely if there is none to disclose.

References

Castelli, F., & Kurucz, R. L. (2003). New Grids of ATLAS9 Model Atmospheres. In N. Piskunov, W. W. Weiss, & D. F. Gray (Eds.), *Modelling of stellar atmospheres* (Vol. 210, p. A20). https://doi.org/10.48550/arXiv.astro-ph/0405087

Jermyn, A. S., Bauer, E. B., Schwab, J., Farmer, R., Ball, W. H., Bellinger, E. P., Dotter, A., Joyce, M., Marchant, P., Mombarg, J. S. G., Wolf, W. M., Sunny Wong, T. L., Cinquegrana, G. C., Farrell, E., Smolec, R., Thoul, A., Cantiello, M., Herwig, F., Toloza, O., … Timmes, F. X. (2023). Modules for Experiments in Stellar Astrophysics (MESA): Time-dependent Convection, Energy Conservation, Automatic Differentiation, and Infrastructure. *265*(1), 15. https://doi.org/10.3847/1538-4365/acae8d

Mészáros, S., Bohlin, R., Allende Prieto, C., Cseh, B., Kovács, J., Fleming, S. W., Dencs, Z., Deustua, S., Gordon, K. D., Hubeny, I., Mező, G., & Truszek, M. (2024). The updated BOSZ synthetic stellar spectral library. *688*, A197. https://doi.org/10.1051/0004-6361/202449306

Paxton, B., Bildsten, L., Dotter, A., Herwig, F., Lesaffre, P., & Timmes, F. (2011). Modules for Experiments in Stellar Astrophysics (MESA). *192*(1), 3. https://doi.org/10.1088/0067-0049/192/1/3

Paxton, B., Cantiello, M., Arras, P., Bildsten, L., Brown, E. F., Dotter, A., Mankovich, C., Montgomery, M. H., Stello, D., Timmes, F. X., & Townsend, R. (2013). Modules for Experiments in Stellar Astrophysics (MESA): Planets, Oscillations, Rotation, and Massive Stars. *208*(1), 4. https://doi.org/10.1088/0067-0049/208/1/4

Paxton, B., Marchant, P., Schwab, J., Bauer, E. B., Bildsten, L., Cantiello, M., Dessart, L., Farmer, R., Hu, H., Langer, N., Townsend, R. H. D., Townsley, D. M., & Timmes, F. X. (2015). Modules for Experiments in Stellar Astrophysics (MESA): Binaries, Pulsations, and Explosions. *220*(1), 15. https://doi.org/10.1088/0067-0049/220/1/15

Paxton, B., Schwab, J., Bauer, E. B., Bildsten, L., Blinnikov, S., Duffell, P., Farmer, R., Goldberg, J. A., Marchant, P., Sorokina, E., Thoul, A., Townsend, R. H. D., & Timmes, F. X. (2018). Modules for Experiments in Stellar Astrophysics (MESA): Convective Boundaries, Element Diffusion, and Massive Star Explosions. *234*(2), 34. https://doi.org/10.3847/1538-4365/aaa5a8

Paxton, B., Smolec, R., Schwab, J., Gautschy, A., Bildsten, L., Cantiello, M., Dotter, A., Farmer, R., Goldberg, J. A., Jermyn, A. S., Kanbur, S. M., Marchant, P., Thoul, A., Townsend, R.

H. D., Wolf, W. M., Zhang, M., & Timmes, F. X. (2019). Modules for Experiments in Stellar Astrophysics (MESA): Pulsating Variable Stars, Rotation, Convective Boundaries, and Energy Conservation. *243*(1), 10. https://doi.org/10.3847/1538-4365/ab2241

Rodrigo, C., & Solano, E. (2020). The SVO Filter Profile Service. *XIV.0 Scientific Meeting (Virtual) of the Spanish Astronomical Society*, 182.

Shankar, S., Gully-Santiago, M. A., Morley, C. V., Cao, J., Kaplan, K., Kimani-Stewart, K., & Gonzalez-Argúeta, D. (2024). <Code>gollum</code>: An intuitive programmatic and visual interface for precomputed synthetic spectral model grids. *Journal of Open Source Software*, *9*(100), 6601. https://doi.org/10.21105/joss.06601

Team, T. M. (2025). *Modules for experiments in stellar astrophysics (MESA) release candidate r25.10.1-rc1* (Version 25.10.1rc1). Zenodo. https://doi.org/10.5281/zenodo.17426065

The MESA Team. (2026). MESA colors module documentation. In *GitHub repository*. GitHub. https://github.com/MESAHub/mesa/blob/08a30a014dbfdaa03e5e046fbd59e9b58a6b26ef/colors/README.rst